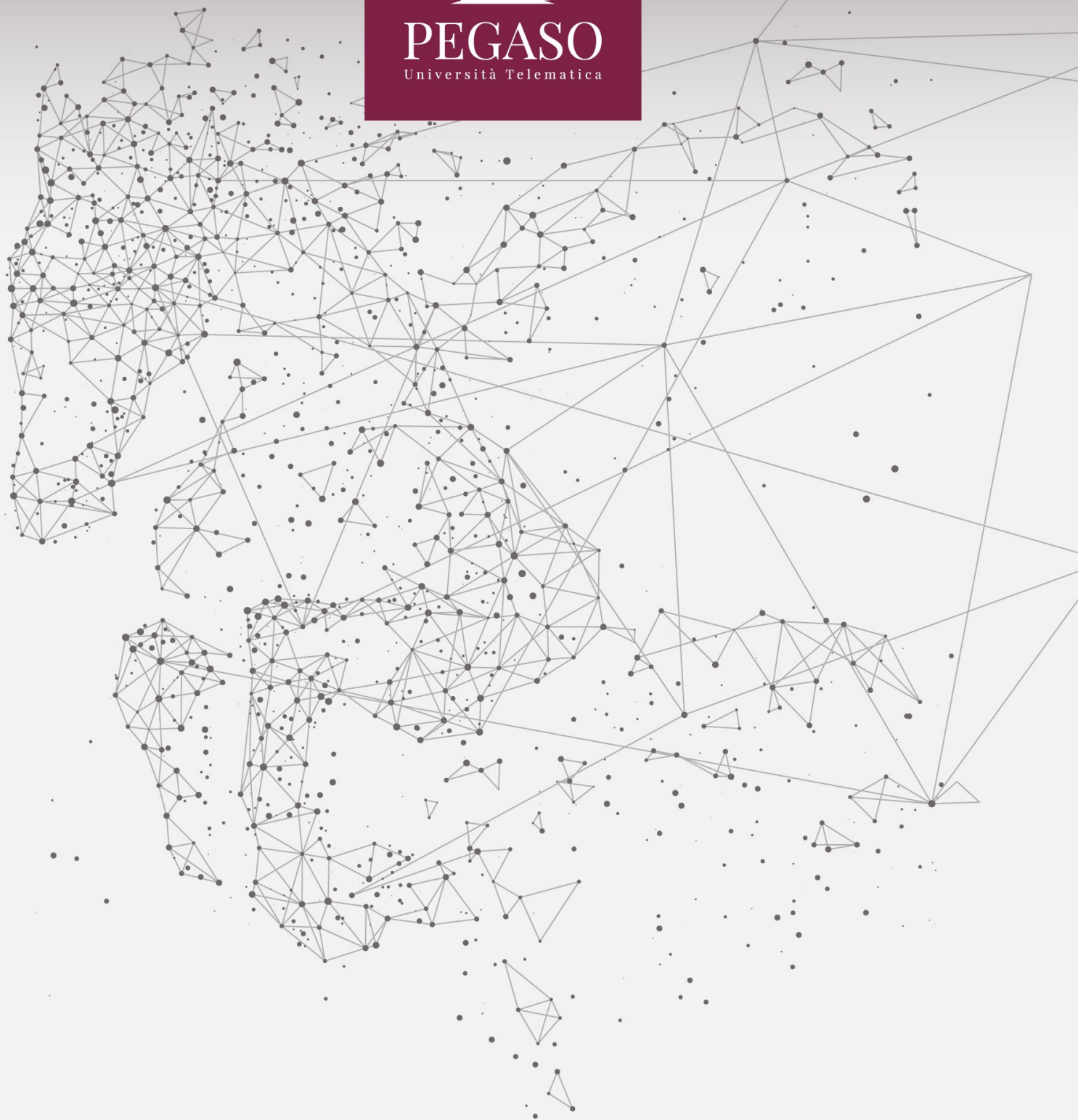




PEGASO
Università Telematica



Indice

1. RAPPRESENTAZIONE DI UN ALGORITMO	3
2. FLOWCHART	4
3. PSEUDOCODICE	9
4. TOOLS	11
BIBLIOGRAFIA	12

1. Rappresentazione di un algoritmo

Nel corso degli anni sono stati sviluppati molti linguaggi di scrittura algoritmi, fra i quali i più noti ed usati sono:

- linguaggio dei diagrammi di flusso (flowchart);
- pseudocodice.

Alcune volte tendiamo a confondere il concetto di algoritmo con il concetto di programma: questi due non sono la stessa cosa; gli algoritmi non sono programmi. I programmi sono un modo di descrivere gli algoritmi, ed in alcuni casi lo sono in modo poco efficiente perché in alcune circostanze, il linguaggio nasconde sia l'intuizione sia di idea che soggiace all'algoritmo stesso.

A tal proposito è bene ricordare la tesi di Church-Turing:

“se un problema è umanamente calcolabile, allora esisterà una macchina di Turing in grado di risolverlo (cioè di calcolarlo)”

Una formulazione alternativa nel nostro contesto di analisi può essere: tutti i linguaggi sufficientemente espressivi sono ugualmente espressivi, vale a dire: possiamo scegliere un linguaggio qualunque, in particolare possiamo utilizzare uno pseudocodice che assomigli il più possibile ai linguaggi classici (e dunque svincolarci dallo specifico linguaggio di programmazione).

L'utilizzo di uno pseudocodice o di un flow-chart ci consente dunque di focalizzarci sulla nostra capacità di sviluppare una “intuizione algoritmica”, cioè trovare possibili soluzioni a problemi, ed in particolare, soluzioni “ottimali”. Analizzare un insieme di problemi ed un insieme di soluzioni buone (ed in alcuni casi ottime) sono importanti per poter costruire un vocabolario mentale per il nostro futuro come informatici.

2. Flowchart

Il diagramma di flusso (flowchart) è una rappresentazione grafica delle operazioni da eseguire per l'esecuzione di un algoritmo. Ogni singolo passo è visualizzato tramite una serie di simboli standard.

Esso consente di descrivere tramite un linguaggio di modellazione grafico:

- le operazioni da compiere, rappresentate mediante sagome convenzionali (rettangoli, rombi, esagoni, parallelogrammi, rettangoli smussati...), ciascuna con un preciso significato logico e all'interno delle quali un'indicazione testuale descrive tipicamente l'attività da svolgere;
- la sequenza nella quale devono essere compiute, è rappresentata con frecce di collegamento.

Per questa loro connotazione topologica i diagrammi di flusso possono essere ricondotti alla classe più ampia dei diagrammi a blocchi, che a loro volta rientrano nell'ancora più vasta categoria delle mappe concettuali, utilizzate soprattutto per la descrizione e la rappresentazione delle informazioni e della conoscenza.

Esistono varie notazioni per la rappresentazione con diagrammi di flusso. Tutte le notazioni sottendono a un meta-modello molto semplice, caratterizzato da una lettura sequenziale:

1. si parte dal blocco iniziale (inizio);
2. si segue la freccia in uscita;
3. si giunge al blocco successivo e si effettua l'operazione descritta nel blocco;
4. si procede iterando i passi 2 e 3 fino a giungere al blocco finale (fine).

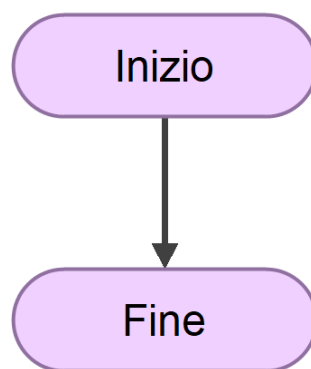


Figura 1 - Inizio e Fine

Un comando di **Output** (scrivi) valuta un'espressione e poi visualizza il risultato sullo schermo:

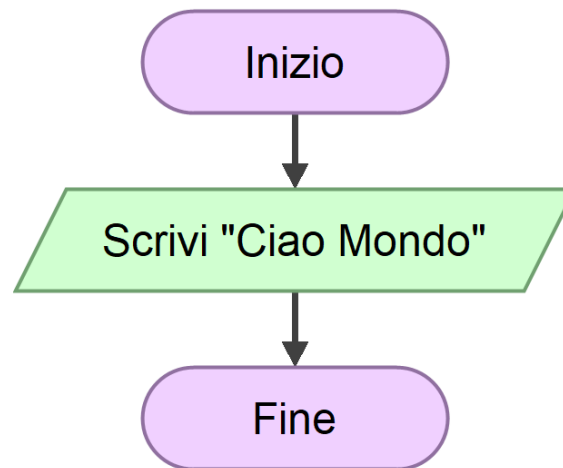


Figura 2 - Scrivi "Ciao Mondo"

Un comando di **Input** (leggi) legge un valore da tastiera e immagazzina tale valore in una variabile; si ha dunque la necessità di definire prima una “variabile” in cui poter immagazzinare il valore inserito:

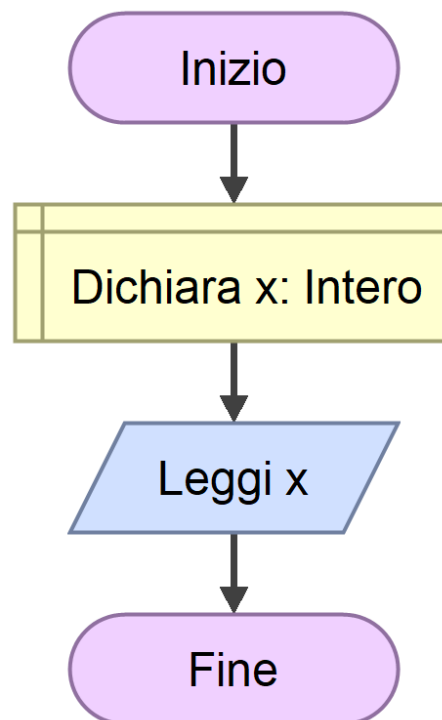


Figura 3 - Lettura input

In questo esempio si è dichiarata una variabile intera “x” e si inserisce l’input da tastiera nella variabile stessa.

Sequenza: le istruzioni sono eseguite secondo l'ordine in cui sono scritte

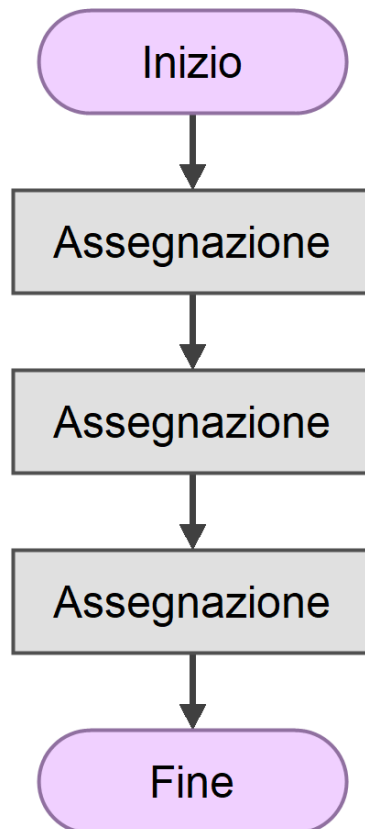


Figura 4 - Sequenza

Selezione o Condizione: esiste una condizione da valutare e due possibili gruppi di istruzioni da eseguire; in funzione del valore della condizione, si sceglie un blocco oppure l'altro

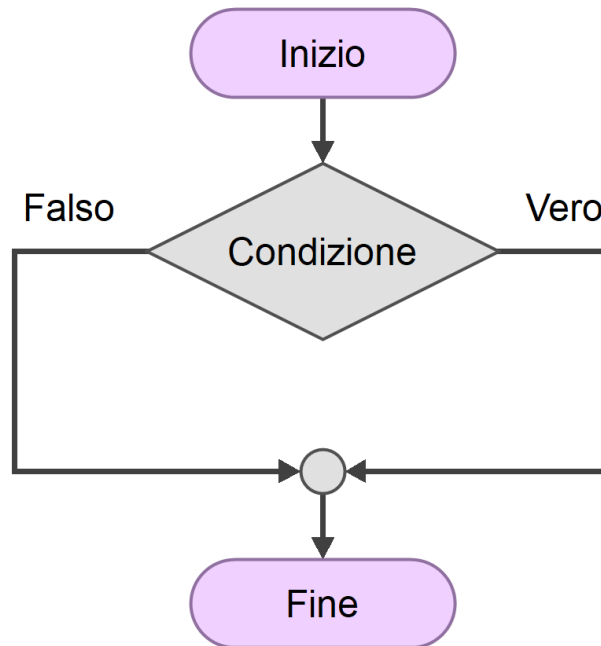


Figura 5 - Selezione

Iterazione: consente la ripetizione di un blocco di istruzioni, per un certo numero di volte, definito a seconda dell'obiettivo da raggiungere.

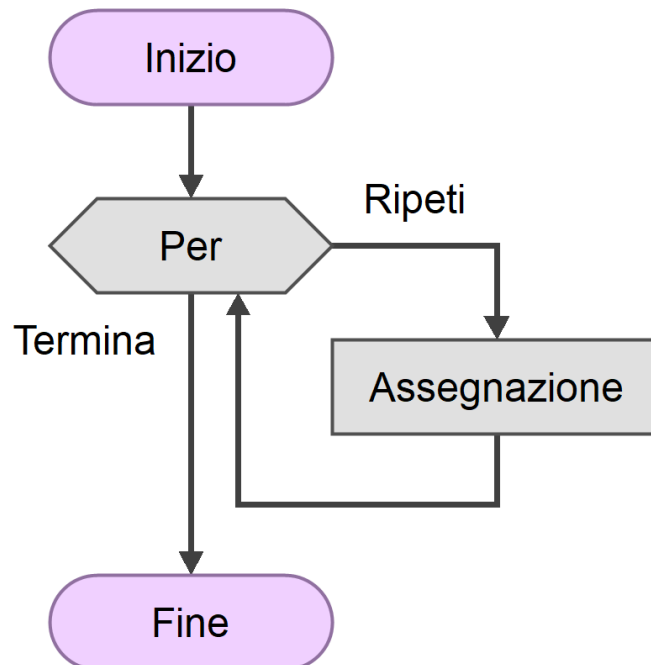


Figura 6 - Iterazione

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Assegnazione: istruzione utilizzata per modificare il contenuto di una variabile

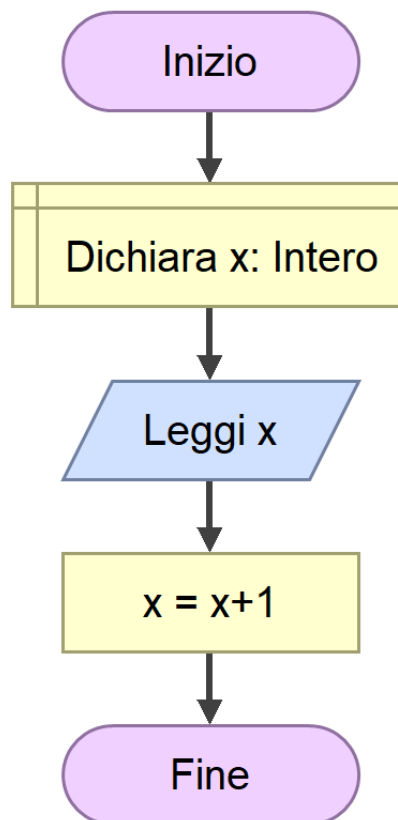


Figura 7 – Assegnazione

In questo caso, dopo aver memorizzato l'input da tastiera nella variabile x, viene modificata la variabile x con il suo valore a cui è sommato 1.

3. Pseudocodice

Non esiste uno pseudolinguaggio standard e convenzionalmente usato: gli autori di libri o corsi di programmazione definiscono spesso un proprio pseudolinguaggio, utilizzato nelle loro pubblicazioni; inoltre, ciascun programmatore può essere portato ad utilizzare una propria variante. Ogni pseudolinguaggio ha un proprio lessico, una propria sintassi e una propria semantica, ma la progettazione di questo tipo di formalismo è volta alla comprensibilità e alla leggibilità del codice; la sintassi sarà quindi meno rigorosa rispetto ad un vero linguaggio e le parole chiave saranno evocative, in modo da rendere più intuitiva la sua interpretazione.

Lo pseudolinguaggio è strettamente dipendente dal paradigma di programmazione scelto per risolvere un problema, mentre dovrebbe essere pressoché indipendente dal linguaggio di programmazione, purché quest'ultimo rispetti naturalmente il paradigma scelto. Tuttavia, ciascun linguaggio di programmazione possiede istruzioni e/o caratteristiche proprie, che potrebbero essere sfruttate per una migliore implementazione dell'algoritmo, ad esempio più efficiente.

Facciamo riferimento alla guida allo “pseudocodice” presenta nel portale AICA ed utilizzato durante le Olimpiadi di Informatica¹.

Di seguito il “cheat sheet” con i punti salienti dello pseudo-codice; nella prima colonna è contenuto lo pseudo-codice, nella seconda la descrizione e nella terza un esempio:

■ Variabili e tipi

variable <i>i</i> : <i>integer</i>	Dichiarazione della variabile di tipo intero chiamata <i>i</i>	
variable <i>arr</i> : <i>integer[]</i>	Dichiarazione di una variabile di tipo array di interi chiamata <i>arr</i>	
{var} ← {espr}	Assegnamento del valore dell'espressione {espr} alla variabile {var}	<i>i</i> ← 1 <i>a</i> ← 3 × <i>i</i> + 5 <i>arr</i> ← [3, 5/ <i>a</i> , 2]
<i>arr</i> [{espr}]	Variabile corrispondente all'elemento dell'array <i>arr</i> di indice {espr}	<i>a</i> ← <i>arr</i> [3] + 1 <i>arr</i> [<i>x</i> + 1] ← 2
(<i>a</i> , <i>b</i>) ← (<i>b</i> , <i>a</i>)	Scambio del valore delle variabili <i>a</i> e <i>b</i>	

■ Operatori

+, −, ×, /, mod	Aritmetica: addizione, sottrazione (o negazione), moltiplicazione, divisione intera, resto della divisione intera (modulo)	<i>a</i> + <i>b</i> − <i>a</i> <i>i</i> ← <i>a</i> mod 10
=, ≠, <, ≤, >, ≥	Confronto: uguale, diverso, minore, minore o uguale, maggiore, maggiore o uguale	<i>a</i> = 7 2 × (<i>x</i> + 1) ≤ <i>y</i>
and , or , not	Operatori logici: e, o, non	<i>a</i> > <i>b</i> and <i>b</i> ≠ −1 not (<i>a</i> > 2 or <i>a</i> == 0)

¹ <https://www.olimpiadi-informatica.it/images/Pseudocodice.pdf>

■ Strutture di controllo

<pre>if {condizione} then {corpo if} else {corpo else} end if</pre>	Struttura condizionale if ... else: se {condizione} è vera viene eseguito {corpo if}, altrimenti viene eseguito {corpo else}. La parte else può essere omessa	<pre>if n mod 2 == 0 then i ← 0 else i ← n - 1 end if</pre>
<pre>while {condizione} do {corpo} end while</pre>	Ciclo while: il blocco {corpo} viene ripetuto fintanto che {condizione} è vera	<pre>while i < n do sum ← sum + i i ← i + 7 end while</pre>
<pre>for {indice} in {intervallo} do {corpo} end for</pre>	Ciclo for: il blocco {corpo} viene eseguito mentre la variabile {indice} itera sui valori in {intervallo}, specificato come [a ... b), che significa “tutti i numeri da a (incluso) fino a b (escluso)”	<pre>for i in [0 ... n) do arr[i] ← -1 end for</pre> (assegna -1 a tutti gli elementi di un array arr di lunghezza n)

■ Funzioni

<pre>function fun(var1: tipo1, var2: tipo2, ...) → ritorno {corpo} end function</pre>	Funzione con parametri var1, var2, etc. Il tipo di ritorno → ritorno può essere omesso. Il valore viene restituito tramite la parola chiave return	<pre>function add(a: integer, b: integer) → integer return a + b end function</pre>
<pre>fun() fun(arg1, arg2, ...)</pre>	Chiamata alla funzione fun (rispettivamente senza argomenti e con argomenti). La funzione output stampa il valore di una variabile oppure una stringa fissata. Le funzioni min e max restituiscono risp. il minimo e il massimo di due interi	<pre>return add(a, b) m ← very_big_integer() output(x) output("string") min(a, b) max(a, b)</pre>

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Tools

Il tool di riferimento per la realizzazione di flowchart sarà: Flowgorithm².

Flowgorithm è un linguaggio basato su semplici diagrammi di flusso. Flowgorithm può convertire in modo interattivo il diagramma di flusso in oltre 18 linguaggi di programmazione (C#, C++, Java, JavaScript, Lua, Perl, Python, Ruby, Swift...).

I precedenti schemi grafici sono stati realizzati tramite Flowgorithm.

² <http://www.flowgorithm.org/>

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione.
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione.
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley.
- Sedgewick: Algoritmi in C, Pearson, 2015.